

第八章 针对连续动作的深度 Q 网络

核心问题：表格型 Q-learning 解决离散状态、离散动作（此处“离散”表示数量少、可数）。

DQN 解决连续状态、离散动作（此处“连续”表示数量多、不可数）。

还有一种情况：连续状态、**连续动作**，需要我们去解决。

因为上面的 Q-learning 以及 DQN，在针对特定状态 s 选择动作时，都是将所有的 $Q(s, a_i)$ 列出来比较大小；但当动作也变为连续时，这一过程实际上无法实现（动作数量无穷多，无法穷举）。

下面介绍几种方案。

对动作进行采样

顾名思义，很常见的**把连续问题转换为离散问题**的思路，很容易理解。

优点是易于操作，缺点是可能 **不精确，可能错过最优解**。

梯度上升

我们实际上想找 $a^* = \arg \max_{a \in \mathcal{A}} Q_\theta(s, a)$ 。注意到在 θ 和 s 固定的情况下， $Q_\theta(s, a)$ 就是一个只关于 a 的函数： $f(a) = Q_\theta(s, a)$ ，求最大值也即 $\max_a f(a)$ 。

至此，**转化为一个简单的一元函数最优化问题，采用梯度上升的方式更新 a** ：

$$a_{k+1} = a_k + \eta \frac{\partial Q_\theta(s, a_k)}{\partial a}$$

这种方法看上去很好，但实际上问题很多：

1. 只能找到**局部最优解**。
2. **计算量很大**：离散 DQN 只需一次前向传播就能算出一个 s 对应的所有 $Q(s, a)$ ，比较大小即可；而连续情况下需要重复执行以下循环 k 次：前向传播算出 $Q(s, a_i)$ ，求对 a 的梯度，更新到 a_{i+1} （其中 k 为从初始动作迭代到“最优”动作的迭代次数）。
3. 对于步长选择 η 、初始动作 a_0 的选择都很敏感。
4. Q 函数的**错误估计会被放大**：和传统 DQN 发生严重高估的原因一样：只要有错误高估的情况出现，大概率就会选到这个高估项，导致错误估计放大。

设计网络架构

把 Q 函数设计成一个特殊的形式：

$$Q(s, a) = -(a - \mu(s))^T \Sigma(s)(a - \mu(s)) + V(s)$$

其中下面这三个量就是网络学习的以及输出：

- $\mu(s)$ ：一个动作向量，最终会成为最优动作；
- $\Sigma(s)$ ：一个正定矩阵，用来决定 Q 函数对动作的“惩罚形状”；
- $V(s)$ ： Q 函数在最高点的数值。

根据数学推导，我们可以知道 $\mu(s)$ 就是最优动作，此时 Q 最大。

几何上，这个 Q 函数可以理解成一个倒扣的椭圆碗， $\mu(s)$ 对应极大值。

这样做的好处：不用再费劲找最优动作。

坏处：**大大限制了 Q 函数的表达能力，难以拟合和“倒扣的碗”相差大的情况。**